

GCD

Name: Rakesh Mandal

Roll No.: CrS2512

Subject: Cryptography and Security Implementation

July 23, 2026

1 Lemma

For any $a, b \in \mathbb{N}$ with $b \neq 0$,

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

2 GCD Algorithm:

Assume $a, b \in \mathbb{N}$ with $a \geq b > 0$

```
unsigned int gcd(unsigned int a, unsigned int b)
{
    while (b != 0)
    {
        unsigned int r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

The above Lemma proves the correctness of the algorithm.

3 Termination of the Algo:

Let $r_0 = a$ and $r_1 = b$. Define the sequence of remainders:

$$r_0 = a, \quad r_1 = b \quad \text{where } a \geq b > 0.$$

Then for $i \geq 1$:

$$r_{i-1} \equiv r_{i+1} \pmod{r_i}.$$

Hence:

$$r_1 > r_2 > \cdots > r_k > 0.$$

Thus, the sequence of remainders is a strictly decreasing sequence of non-negative integers. Since a sequence of non-negative integers cannot decrease indefinitely, there exists some k such that $r_k = 0$.

Therefore, the algorithm always terminates after finitely many iteration.

4 Total Number of Iterations:

Assume a, b are n -bit non-negative integers. Then

$$0 < a, b < 2^n.$$

Claim 1. *For all $i \geq 1$, $r_{i+2} < \frac{r_i}{2}$.*

Proof. Case 1: If $r_{i+1} \leq \frac{r_i}{2}$. Since $\{r_i\}$ is a strictly decreasing sequence, we have:

$$r_{i+2} < r_{i+1} \leq \frac{r_i}{2} \implies r_{i+2} < \frac{r_i}{2}.$$

Case 2: Suppose $r_{i+1} > \frac{r_i}{2}$. Then,

$$r_i = q_{i+1}r_{i+1} + r_{i+2}$$

Since $r_{i+1} > \frac{r_i}{2}$, it follows that,

$$\begin{aligned} r_i < 2r_{i+1} &\implies r_{i+1} < r_i < 2r_{i+1} \\ &\implies 1 < \frac{r_i}{r_{i+1}} < 2 \\ &\implies q_{i+1} = 1 \\ &\implies r_i = r_{i+1} + r_{i+2} \\ &\implies r_{i+2} = r_i - r_{i+1} \end{aligned}$$

Since $r_{i+1} > \frac{r_i}{2}$, we obtain

$$r_{i+2} < r_i - \frac{r_i}{2} = \frac{r_i}{2}.$$

□

Claim 2. *The algorithm terminates after at most $2n$ iterations*

Proof. Initially, $r_1 = b < 2^n$. By Claim 1, after $2k$ iterations, we get

$$r_{2k+1} < \frac{r_1}{2^k} < \frac{2^n}{2^k} = 2^{n-k}.$$

Choosing $k = n$, we get

$$r_{2n+1} < 2^{n-n} = 2^0 = 1.$$

Since r_{2n+1} must be a non-negative integer, this implies:

$$r_{2n+1} = 0.$$

Hence, the algorithm terminates after at most $2n$ iterations.

□